

Give an LLM the web with a first *agent loop*

You already called a simple tool. Now: Tavily Search + Extract inside a Python loop — the primitive under competitor research, not the finished agent.

LEARNING OUTCOME

By the end, you can expose Search and Extract as typed tools and run a Python tool-calling loop that answers current-fact questions with cited sources.



tavily

+



Nebius Token Factory

Competitor research cannot run on **frozen knowledge**

COURSE PROJECT

A competitor matrix goes stale in weeks

Pricing changes. Features ship. Funding lands. Your agent has to fetch current facts — not invent them.

- Who competes with us right now?
- What do they charge this quarter?
- What shipped since the model was trained?

KNOWLEDGE CUTOFF

Nemotron 3 Nano

June 25, 2025

Pre-training data freshness on Hugging Face. After that date, retrieve — don't guess.

If a fact can change after the cutoff, require a URL before you trust the answer.

The model reasons. The web supplies the **evidence**.

Ask a model (no search): “What were the biggest AI model releases this month?”

LLM

REASONING ENGINE

Strong reasoning, no new facts

- Plans, compares, and synthesizes
- Reasons only over what it can see
- Cannot fetch what changed since training

WEB

EVIDENCE SOURCE

Current, citable, checkable

- News, docs, releases, and prices
- URLs you can verify and cite
- Fresh context the model can reason over

Division of labor: reasoning is the model's job; current facts are the web's job. Tools connect the two.

Search finds sources. Extract reads them.

01

DISCOVER

Search

Ask a question. Get ranked pages — title, URL, query-relevant snippet, and a relevance score.

QUERY → SOURCES

02

READ

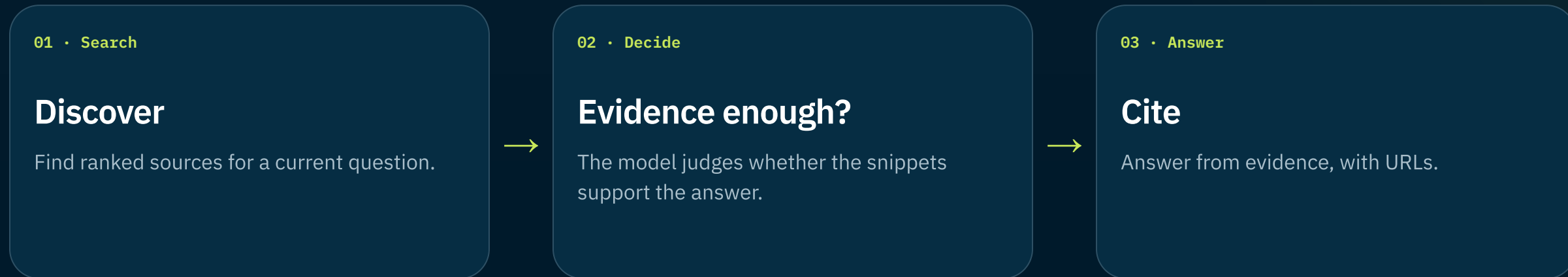
Extract

Pass one or more URLs. Get the cleaned full page as markdown or text.

URLS → CONTENT

Search first to shortlist. Extract only when the snippet cannot support the claim.

Two tools. One loop.

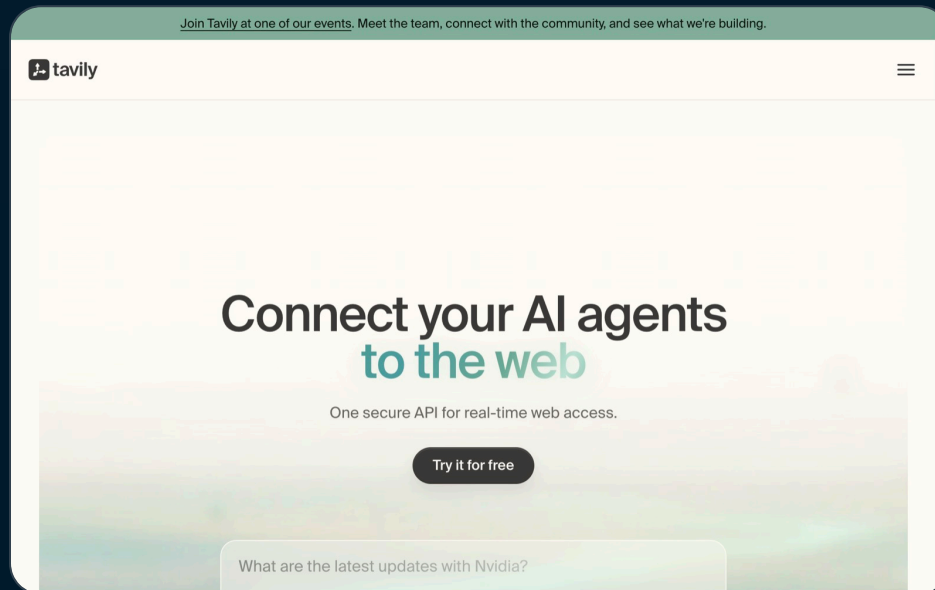


🔄 Not enough? The model picks the next step — **Extract** a promising URL or **Search again** — then decides once more. Extract is optional; any step can repeat, in any order.

Choosing — and repeating — the next step is what makes this an agent loop, not a predefined pipeline.

Sign up for Tavily — free API key in two minutes

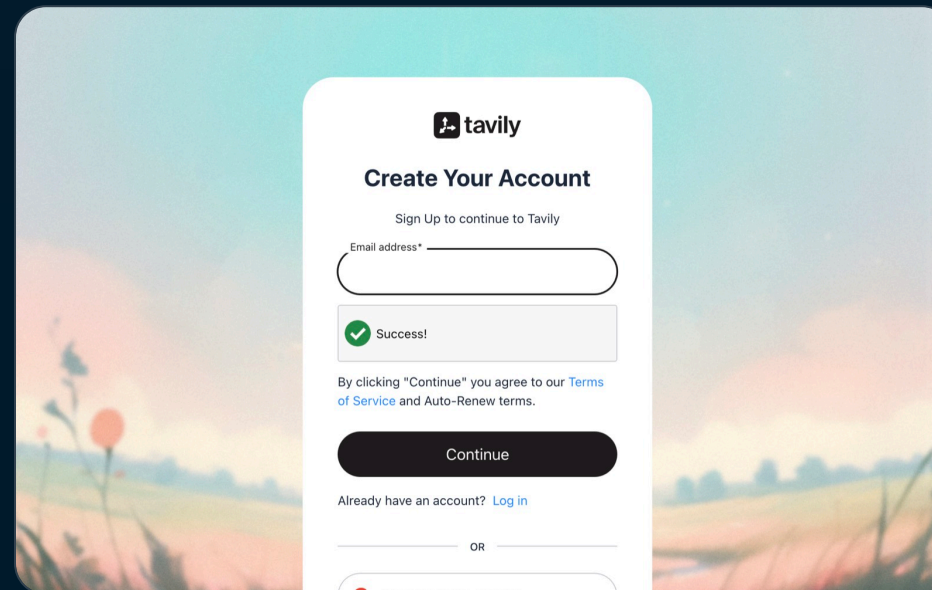
01 OPEN



Go to tavily.com

Click [Try it for free](#).

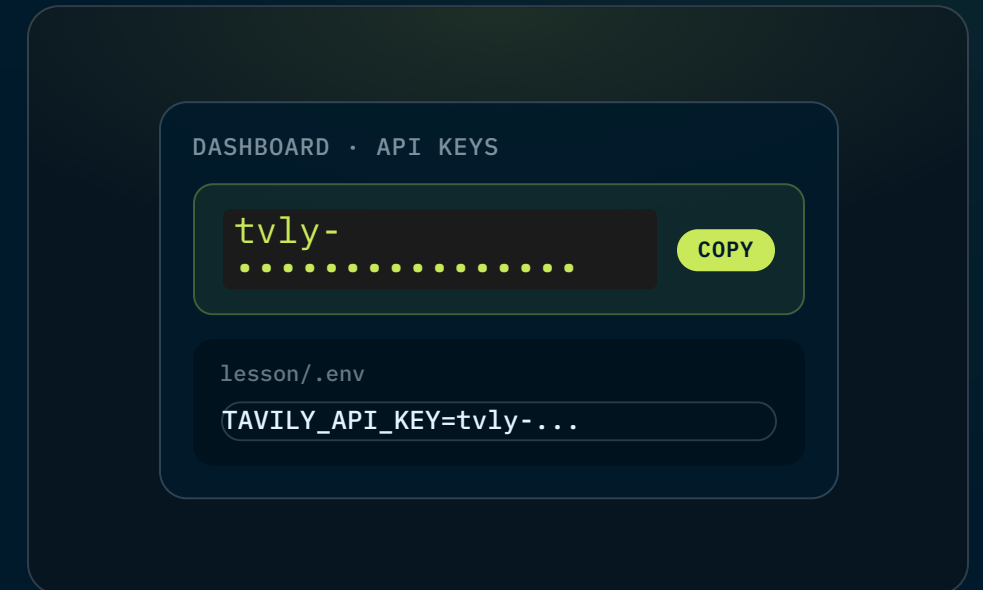
02 SIGN UP



Create an account

Email, or continue with Google / GitHub.

03 COPY KEY



Paste into `.env`

1,000 free credits / month — no credit card.

Put `TAVILY_API_KEY` in `lesson/.env` before you run the notebook. Keys always start with `tvly-`.

Underneath, every tool is an HTTP call

```
response = requests.post(
    "https://api.tavily.com/search",
    headers={"Authorization": f"Bearer {TAVILY_API_KEY}"},
    json={
        "query": "biggest AI model releases this month",
        "search_depth": "advanced",
        "max_results": 5,
        "time_range": "month",
    },
    timeout=30,
)
data = response.json()
```

ENDPOINT

POST /search

SEND

query + runtime parameters

RECEIVE

Scored results with URLs and snippets

NEXT

POST /extract on chosen URLs

SDKs cut boilerplate. They do not change the model: request in, structured evidence out.

What Search returns to the model

01 RESULT

ILLUSTRATIVE EXAMPLE

0.96

Latest AI Model Releases — July 2026

<https://example.com/ai-model-releases-july-2026>

Model Alpha (Jul 16), Model Beta (Jul 9), Model Gamma (Jul 8) — a query-focused snippet so the model can skim without loading the full page.

title What the page claims to cover

url What you cite — or pass to Extract

content Snippet ranked for this query

score Relative relevance, 0 → 1

These fields are small enough for context. Full pages wait for Extract — or never load.

A tool is a function the model can request

```
def internet_search(
    query: str,
    search_depth: str = "advanced",
    max_results: int = 5,
) -> str:
    """Search the web for current, factual sources."""
    data = tavily_client.search(
        query,
        search_depth=search_depth,
        max_results=max_results,
    )
    return format_results(data["results"])

def extract_content(
    urls: list[str],
    query: str | None = None,
) -> str:
    """Read full pages when snippets are not enough."""
    data = tavily_client.extract(
        urls=urls,
        query=query,
        extract_depth="advanced",
        format="markdown",
    )
    return format_extractions(data)
```

Clear purpose

Name and docstring tell the model when to use it.

Typed inputs

Arguments become the schema the model must follow.

Compact output

Return titles, URLs, and evidence — not raw API objects.

Test the function like any other Python. Only then wire it into an agent loop.

Same schema pattern — now for Tavily

You saw tool calling earlier. Here is Search exposed the same way.

```
def internet_search(
    query: str,
    search_depth: str = "advanced",
    max_results: int = 5,
) -> str:
    ...
```



Your app must call this function directly.

The model never reads the Python source.

```
{
  "type": "function",
  "function": {
    "name": "internet_search",
    "description": "Search the web for current, factual information. Use this first to discover sources.",
    "parameters": {
      "type": "object",
      "properties": {
        "query": { "type": "string" },
        "search_depth": {
          "type": "string",
          "enum": ["basic", "fast", "advanced"]
        },
        "max_results": { "type": "integer" }
      },
      "required": ["query"]
    }
  }
}
```

name — which function to invoke

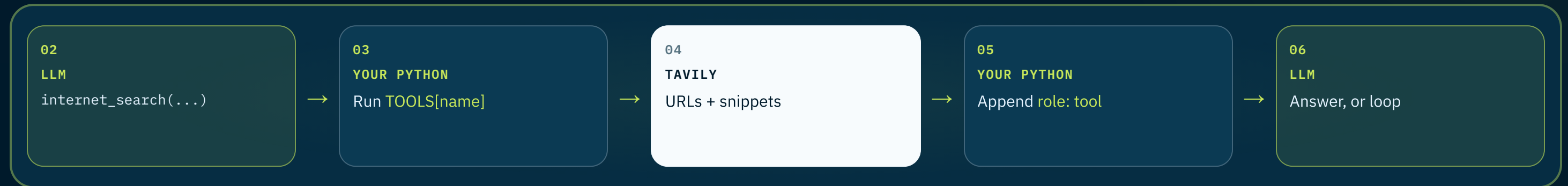
description — when to choose Search vs Extract

parameters — what arguments are valid

Write the description as policy: use Search first; Extract only when snippets are thin.

The model proposes. Your code runs.

01 • USER What were the biggest AI model releases this month?



🔄 If the model emits more `tool_calls`, return to step 02 — that is the agent loop.

The LLM never runs Python. It emits a structured request; the application chooses what to execute.

A minimal agent is a control loop

```
def run_agent(question: str, max_steps: int = 10):
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": question},
    ]
    for step in range(1, max_steps + 1):
        msg = client.chat.completions.create(
            model=MODEL,
            messages=messages,
            tools=AGENT_TOOLS,
        ).choices[0].message
        messages.append(msg.model_dump(exclude_none=True))

        if not msg.tool_calls:
            return msg.content

        for tc in msg.tool_calls:
            args = json.loads(tc.function.arguments)
            result = TOOLS[tc.function.name](**args)
            messages.append({
                "role": "tool",
                "tool_call_id": tc.id,
                "content": result,
            })
    return "Stopped: reached max_steps."
```

01 Ask the model

02 Execute tools

03 Append results

04 Answer or repeat

AGENT LOOP

Stop when there are no `tool_calls`, or when `max_steps` is hit.

This is the loop DeepAgents will wrap next. Frameworks do not replace it.

What this minimal loop still lacks

01

No plan

It reacts call-by-call instead of decomposing a hard question up front.

02

Duplicate work

Nothing stops a repeated query or a second extract of the same URL.

03

Blind ranking

A high score is not the same as an authoritative source.

04

Context bloat

Full pages pile into message history until the window is full.

Next: Orchestrating deep research — turn this loop into a full search pipeline with DeepAgents.

— SUMMARY

Key ideas from *this module*

Two retrieval tools plus a Python-owned loop — the primitive under competitor research.

01

Search

Discover sources

02

Extract

Read full pages

03

Schema

Expose the tool

04

Loop

Call until done

The model proposes a tool call → Python executes it → the result returns to the model → repeat until an answer.